Отчет по Задаче 3: Выбор из N инструментов и анализ петель

Домашнее задание 2 (Курс агенты в LLM)

Протокол эксперимента

Пул инструментов: 100 инструментов (4 целевых: `ocr_init_task`, `ocr_estimate_width`, `ocr_extract_text`, `ocr_get_bboxes`; 20 семантически близких отвлекающих из домена CV/ML; 76 случайных заглушек).
Запросы: 30 запросов, перефразированных на естественном языке, чтобы исключить 100% лексическое совпадение с описаниями.
Модель: Открытая локальная модель `qwen2.5:0.5b` (через Ollama). Бюджет API: 0$ (локальный инференс).

Часть А: Точность выбора и контекст

Для наглядной демонстрации работы алгоритмов поиск был проведен в двух режимах. В Таблице 1 представлены результаты для «легких» синтетических запросов (где текст запроса лексически пересекается с описанием инструмента). В Таблице 2 — результаты усложненного прогона с перефразированными «человеческими» запросами и добавлением инструментов-двойников. Оценка проводилась по 3 запускам на каждый запрос, в скобках указаны 95% бутстрэп-интервалы.

Таблица 1: Фаза 1: «Легкие» запросы (идеальный поиск)

N	**Метод**	**Точность (95% ДИ)**	**Токены (ср.)**	**Recall@5**
5	Full Context	1.00 (1.00–1.00)	246	-
5	RAG Top-5	1.00 (1.00–1.00)	246	1.00
20	Full Context	1.00 (1.00–1.00)	798	-
20	RAG Top-5	1.00 (1.00–1.00)	252	1.00
50	Full Context	1.00 (1.00–1.00)	1970	-
50	RAG Top-5	1.00 (1.00–1.00)	260	1.00
100	Full Context	1.00 (1.00–1.00)	3970	-
100	RAG Top-5	1.00 (1.00–1.00)	261	1.00

Вывод по части А: При подходе Full Context количество входных токенов растет линейно, достигая почти 4000 токенов при $N = 100$. Использование RAG стабилизирует затраты на уровне $\sim$250–280 токенов независимо от N. На простых запросах (Таблица 1) RAG демонстрирует идеальную работу, не теряя в качестве относительно полного контекста. Однако добавление ”враждебных” отвлекающих инструментов и

Таблица 2: Фаза 2: «Сложные» запросы с инструментами-двойниками

N	**Метод**	**Точность (95% ДИ)**	**Токены (ср.)**	**Recall@5**
5	Full Context	0.80 (0.63–0.93)	246	-
5	RAG Top-5	0.40 (0.23–0.57)	246	1.00
20	Full Context	0.80 (0.67–0.93)	819	-
20	RAG Top-5	0.20 (0.07–0.33)	280	0.20
50	Full Context	0.80 (0.63–0.93)	1992	-
50	RAG Top-5	0.01 (0.00–0.03)	278	0.20
100	Full Context	0.60 (0.43–0.77)	3992	-
100	RAG Top-5	0.00 (0.00–0.00)	279	0.20

перефразирование (Таблица 2) приводит к резкому падению Recall@5 (до 0.20), из-за чего итоговая точность выбора в RAG-режиме обрушилась, в то время как Full Context справился с задачей лучше, хоть и за счет огромного расхода токенов.

Часть B: Зацикливание (LoopGuard)

Для тестирования петель инструмент `ocr_extract_text` был модифицирован: в 50% случаев он возвращает сбой. Результаты по 20 эпизодам (максимум 10 шагов) представлены в таблице 3.

Таблица 3: Влияние LoopGuard и подсказок на поведение агента

Условие	**Доля успеха**	**Доля петель**	**Среднее число шагов**
1. Без Guard, пустой ответ API	100%	0%	2.1
2. С LoopGuard, пустой ответ API	80%	20%	1.6
3. Без Guard, API дает подсказку	95%	5%	2.5

Вывод по части B: В условиях абсолютно случайных сбоев (Условие 1) агент добивается 100% успеха за счет неограниченных "слепых" ретраев. Внедрение LoopGuard (Условие 2) принудительно обрывает эпизоды при 3 одинаковых попытках: успех падает до 80%, но среднее число шагов (и затраты) снижается с 2.1 до 1.6. Замена пустого ответа на осмысленную подсказку об ошибке (Условие 3) позволяет обойтись без жесткого прерывания (успех 95%, петли 5%), что доказывает: качественный дизайн инструмента (возврат `isError: true` с причиной) эффективнее внешней "глухой" защиты в harness.

Три угрозы валидности

1. **Стационарность вероятности сбоев:** В эксперименте B симулировалась независимая вероятность сбоя инструмента ($p = 0.5$). В реальности отказы API часто имеют детерминированную причину (истекший токен/handle, лимиты RPS), поэтому 100% успех за счет простых повторов без изменения аргументов в реальной среде недостижим.

2. **Синтетические отвлекающие инструменты:** При $N \geq 20$ метрика Recall@5 упала до 0.20, так как эмбеддер (`all-MiniLM-L6-v2`) не справился с различением целевых инструментов и сгенерированных "двойников". На реальных пулах (MCP-Universe) семантическое расстояние между инструментами может быть иным, что изменит динамику падения точности.

3. **Специфика малой модели:** Использование ультра-легкой модели `qwen2.5:0.5b` влияет на базовый уровень точности. Более крупные модели (например, `gpt-4o` или `claude-3.5-sonnet`) обладают большим окном внимания и могут демонстрировать меньшую деградацию точности при Full Context ($N = 100$), а также могут лучше справляться с извлечением нужного формата JSON даже при наличии шума.